PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage
Bengaluru 560085
Department of Computer Science and Engineering

Department of Computer Science and Engineering
B. Tech. CSE - 6th Semester
Jan – May 2024

UE21CS342BA2
Algorithms for Information Retrieval and Intelligence Web

PROJECT REPORT
on

Law Information Retrieval System

Submitted by : Team #: 16

Hamsini Ramesh	**PES2UG21CS183**	6 D
Isha Prabhu	**PES2UG21CS204**	6 D

Class of Prof. Dr.Alpha Vijayan

Table of Contents

Sl. No	Topic	Page No.
1.	Brief description of the project	3
2.	Functionalities	4
3.	Technology used	6
4.	Code snippet	7
5.	Results	13

Brief description of the project

The Law Information Retrieval System utilising TF-IDF (Term Frequency-Inverse Document Frequency) and Cosine Similarity is designed to efficiently search and retrieve legal documents based on the similarity between user queries and the content of documents.

TF-IDF is a statistical measure that evaluates the importance of a term within a document relative to a collection of documents. It helps in quantifying how relevant a term is in a document. Cosine Similarity, on the other hand, is a metric used to determine the similarity between two vectors by measuring the cosine of the angle between them.

In this system, each legal document is represented as a vector in a high-dimensional space, with each dimension corresponding to a term's TF-IDF score. When a user enters a query, it is also transformed into a vector using the same TF-IDF weighting scheme. Then, the cosine similarity between the query vector and each document vector is calculated. The documents with the highest cosine similarity scores are considered the most relevant to the query and are returned as search results.

This system allows legal professionals and researchers to quickly and accurately find relevant legal documents based on the similarity between their queries and the content of the documents, enhancing the efficiency of information retrieval in the field of law.

Functionalities

1. Search: Users can enter search queries using keywords, phrases, or sentences relevant to their legal research needs.

2. TF-IDF Weighting: The system utilises TF-IDF weighting to assign importance scores to terms within documents and queries, enhancing the relevance of search results.

3. Cosine Similarity Calculation: Cosine similarity is calculated between the query vector and document vectors to measure the similarity between them.

4. Ranking: Search results are ranked based on cosine similarity scores, with documents having higher similarity scores to the query being prioritised.

5. Stemming: Stemming is the process of reducing words to their root or base form, stripping away suffixes and prefixes. By performing stemming on both the documents and the user queries, the system ensures that variations of words (e.g., "running," "ran," "runs") are treated as the same term, thereby increasing the chances of retrieving relevant documents.

6. Lemmatization: Lemmatization is similar to stemming but goes a step further by reducing words to their canonical or dictionary form (lemmas). Unlike stemming, lemmatization considers the context of the word and produces meaningful lemmas. By lemmatizing words in documents and queries, the system can further improve the accuracy of matching and retrieval by reducing ambiguity and ensuring that semantically related terms are treated as equivalent.

7. Phrase Query: This functionality allows users to search for documents containing specific phrases rather than individual keywords. For example, a user searching for "breach of contract" would retrieve documents that contain this exact phrase, ensuring more precise search results.
8. Keyword Search: Users can enter individual keywords relevant to their legal research needs. This functionality provides flexibility, allowing users to quickly search for documents related to specific topics or concepts without the need for complex query construction.
9. Boolean Query: Boolean operators such as AND, OR, and NOT can be used to construct complex search queries. Users can combine keywords, phrases, and other search terms using these operators to retrieve documents that meet

specific criteria. For instance, a user could search for "contract AND breach" to find documents that discuss both contracts and breaches.

10. Search Using Case Title: Users can search for legal documents using the title of a specific case. This feature enables users to quickly locate documents related to a particular legal case, providing context and background information for their research.
11. Wild Card Query Search: Wildcard queries, including prefix and suffix variations, broaden the search capabilities of the Law Information Retrieval System. Prefix queries allow users to find terms beginning with a specified sequence of characters, facilitating searches for words like "con*" to retrieve documents containing terms such as "contract" or "conference." On the other hand, suffix queries enable users to locate terms ending with a specific sequence, as in "*ment," which would yield documents featuring words like "agreement" or "investment." By incorporating these wildcard functionalities, the system enhances users' ability to find relevant legal documents, catering to a wider range of search preferences and needs.

Technology used

1. Python: As a versatile and widely-used programming language, Python serves as the backbone for implementing the system's logic, data processing, and user interface components.
2. Pandas: Pandas is used for data manipulation and analysis, facilitating tasks such as reading and preprocessing datasets, organizing data structures, and performing data transformations.
3. NumPy: NumPy provides essential support for numerical computing, enabling efficient handling of arrays, matrices, and mathematical operations. It's particularly useful for vectorized operations and calculations involved in TF-IDF computation and cosine similarity calculation.
4. NLTK (Natural Language Toolkit): NLTK offers a suite of libraries and tools for natural language processing tasks, including tokenization, stemming, lemmatization, and part-of-speech tagging. It's instrumental in processing textual data from legal documents and user queries.
5. PorterStemmer: PorterStemmer is an implementation of the Porter stemming algorithm, which is used to reduce words to their root or base form. It helps in standardizing terms across documents and queries, improving the accuracy of retrieval.
6. TfidfVectorizer: TfidfVectorizer from scikit-learn is employed to transform text data into TF-IDF vectors. It calculates the TF-IDF scores for terms in documents and queries, producing high-dimensional vectors that represent the importance of terms.
7. CountVectorizer (cv): CountVectorizer, also from scikit-learn, is utilized to convert text data into a matrix of token counts. It's often used in conjunction with TfidfVectorizer for preprocessing steps or as an alternative vectorization method.
8. Cosine Similarity: The cosine_similarity function from scikit-learn computes the cosine similarity between pairs of vectors. It's applied to calculate the similarity between TF-IDF vectors of queries and documents, aiding in ranking and retrieving relevant documents.

Code snippet

Loading data from CSV file

```
import csv
import os
import re
from collections import defaultdict
```

```
# Load data from CSV file
def load_data(file_path):
    documents = []
    with open(file_path, 'r', newline='', encoding='utf-8') as csvfile:
        reader = csv.DictReader(csvfile, delimiter=',')
        for row in reader:
            documents.append(row)
    return documents
documents = load_data("/content/output.csv")
print(documents)
```

```
def tokenize(text):
    return re.findall(r'\w+', text.lower())
```

Function to construct inverted index in a dictionary

```
# Function to index document
def index_documents(documents):
    inverted_index = defaultdict(list)
    doc_ids = []

    for i, doc in enumerate(documents):
        doc_ids.append(i)
        tokens = tokenize(doc['case_title'] + " " + doc['case_text'])
        for token in tokens:
            inverted_index[token].append(i)

    return inverted_index, doc_ids
inverted_index, doc_ids = index_documents(documents)
print(inverted_index, doc_ids)
```

Keyword Search

```
# Function to search by keyword
def keyword_search(query, inverted_index, doc_ids, documents):
    query_tokens = tokenize(query)
    relevant_docs = set(doc_ids)

    for token in query_tokens:
        if token in inverted_index:
            relevant_docs =
relevant_docs.intersection(set(inverted_index[token]))

    if relevant_docs:
        print("Search results:")
        for doc_id in relevant_docs:
            print(f"Case ID: {documents[doc_id]['case_id']}")
            print(f"Case Title: {documents[doc_id]['case_title']}")
            print(f"Case Text: {documents[doc_id]['case_text']}")
            print()
    else:
        print("No matching documents found.")
```

Wildcard query search

```
def wildcard_query_search(inverted_index, documents, query):
    wildcard = query
    wildcard = wildcard.lower()

    # Check if the wildcard is a prefix or suffix
    if wildcard.startswith("*"):
        pattern = re.compile(wildcard.replace("*", ".*") + "$")  # Suffix
wildcard
    elif wildcard.endswith("*"):
        pattern = re.compile("^" + wildcard.replace("*", ".*"))  # Prefix
wildcard
    else:
        print("Invalid wildcard query format. Please use '*' as prefix or
suffix.")
        return

    matching_docs = []
```

```
    for token, doc_ids in inverted_index.items():
        if pattern.match(token):
            matching_docs.extend(doc_ids)

    matching_docs = list(set(matching_docs))  # Remove duplicates
    if matching_docs:
        print("Search results:")
        for doc_id in matching_docs:
            print(f"Case ID: {documents[doc_id]['case_id']}")
            print(f"Case Title: {documents[doc_id]['case_title']}")
            print(f"Case Text: {documents[doc_id]['case_text']}")
            print()
    else:
        print("No matching documents found.")
```

Phrase query

```
def phrase_query_search(documents, query):
    found = False
    for doc in documents:
        text = doc['case_title'] + " " + doc['case_text']
        if re.search(r'\b' + re.escape(query) + r'\b', text):
            print("Search results:")
            print(f"Case ID: {doc['case_id']}")
            print(f"Case Title: {doc['case_title']}")
            print(f"Case Text: {doc['case_text']}")
            print()
            found = True

    if not found:
        print("No matching documents found.")
```

Search using case title

```
def search_title(query, documents):
    found = False
    for doc in documents:
        if query == doc['case_title']:
            print("Search results:")
            print(f"Case ID: {doc['case_id']}")
            print(f"Case Title: {doc['case_title']}")
```

```
            print(f"Case Text: {doc['case_text']}")
            print()
            found = True
            break

    if not found:
        print("No matching documents found.")
```

Boolean query Search

```
def boolean_query_search(query, inverted_index, doc_ids, documents):

    query_tokens = query.split()

    if len(query_tokens) < 3 or len(query_tokens) % 2 == 0:
        print("Invalid query format. Please provide a valid boolean query.")
        return

    relevant_docs = set(doc_ids)
    for i in range(1, len(query_tokens), 2):
        operator = query_tokens[i - 1].upper()
        term = query_tokens[i]
        term_docs = set()

        if term in inverted_index:
            term_docs = set(inverted_index[term])

        if operator == "AND":
            relevant_docs = relevant_docs.intersection(term_docs)
        elif operator == "OR":
            relevant_docs = relevant_docs.union(term_docs)
        elif operator == "NOT":
            relevant_docs = relevant_docs.difference(term_docs)

    # Ensure that only documents containing all query terms are considered
    for doc_id in relevant_docs.copy():
        doc_text = documents[doc_id]['case_title'] + ' ' +
documents[doc_id]['case_text']
        if all(term in doc_text for term in query_tokens[::2]):
            print(f"Case ID: {documents[doc_id]['case_id']}")
            print(f"Case Title: {documents[doc_id]['case_title']}")
```

```
            print()

    if not relevant_docs:
        print("No matching documents found.")
```

Retrieval based on ranking by similarity measure.
Applying stemming

```
import nltk
from nltk.stem.porter import PorterStemmer
ps=PorterStemmer()
def stem(text):
    y=[]
    for i in text.split():
        y.append(ps.stem(i))
    return " ".join(y)
```

Finding out TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
cv=TfidfVectorizer(max_features=5000,stop_words="english")
print(cv.get_stop_words())
cv.fit_transform(new_mov["tags"]).toarray().shape
temp=TfidfVectorizer(max_features=5000,stop_words="english")
temp=temp.fit_transform(new_mov["tags"]).toarray()
temp

import pandas as pd
import nltk
from nltk.corpus import stopwords
nltk.download('stopwords')
nltk.download('punkt')
def remove_stopwords(text):
    stop_words = set(stopwords.words('english'))
    words = nltk.word_tokenize(text)
    return ' '.join([word for word in words if word.lower() not in
stop_words])
new_mov['tags'] = new_mov['tags'].apply(remove_stopwords)
```

Finding out cosine similarity

```
from sklearn.metrics.pairwise import cosine_similarity
sim=cosine_similarity(temp)
sim
```

Defining function to retrieve similar cases

```
def retrieve(case,n):
    case_id=new_mov[new_mov["case_id"]==case].index[0]
    distances=sim[case_id]
    case_list=sorted(list(enumerate(distances)),reverse=True,key=lambda
x:x[1])[1:n+1]
    case = [new_mov.iloc[score[0]]['case_id'] for score in case_list]
    print(case)
```

To retrieve top 5 cases relevant to case 2

```
retrieve("Case2",5)
```

Results

Output Screenshots of various functionalities:

- Loading Data:

```
[42] import csv
     import os
     import re
     from collections import defaultdict
```

```
[43] # Load data from CSV file
     def load_data(file_path):
         documents = []
         with open(file_path, 'r', newline='', encoding='utf-8') as csvfile:
             reader = csv.DictReader(csvfile, delimiter=',')
             for row in reader:
                 documents.append(row)
         return documents
```

```
[44] documents = load_data("/content/output.csv")
     print(documents)
```

```
[{'case_id': 'Case1', 'case_outcome': 'cited', 'case_title': 'Alpine Hardwood (Aust) Pty Ltd v Hardys Pty Ltd (No 2) [2002] FCA 224 ; (2002) 190
```

```
[45] def tokenize(text):
         return re.findall(r'\w+', text.lower())
```

- Inverted index

```
[47] inverted_index, doc_ids = index_documents(documents)
     print(inverted_index, doc_ids)
```

```
defaultdict(<class 'list'>, {'alpine': [0, 0, 2], 'hardwood': [0, 0, 2], 'aust': [0, 0, 2, 82, 83, 89, 89, 129, 129, 144, 144, 145, 272, 272, 27
```

- Keyword Search

```
[49] query = "Australia"
     keyword_search(query, inverted_index, doc_ids, documents)
```

```
Case ID: Case986
Case Title: Forster v Farquhar (1893) 1 QB 564
Case Text: Court's discretion to order costs under s 43 of the Federal Court of Australia Act 1976 (Cth) is unfettered, except that it must be

Case ID: Case987
Case Title: Inn Leisure Industries Pty Ltd v DF McCloy Pty Ltd (No 2) (1991) 28 FCR 172
Case Text: Court's discretion to order costs under s 43 of the Federal Court of Australia Act 1976 (Cth) is unfettered, except that it must be

Case ID: Case998
Case Title: Coco v R (1994) 179 CLR 427
Case Text: applicant relies upon the decision of the Full Court of the Supreme Court of South Australia in Dimella Constructions Pty. Ltd v St

Case ID: Case999
Case Title: Customs and Excise Commissioners v. Air Canada [1991] 2 QB 446
Case Text: answer to the applicant's submissions lie the words used in s 261A of the Migration Act . The words providing for the forfeiture ar

Case ID: Case1000
Case Title: Dimella Constructions Pty. Ltd v Stocker and Stocker (1976) 14 SASR 215
Case Text: applicant relies upon the decision of the Full Court of the Supreme Court of South Australia in Dimella Constructions Pty. Ltd v St

Case ID: Case1003
Case Title: McMullin and Another v ICI Australia Operations Pty Ltd and Others (No 7) [1999] FCA 1814 ; (1999) 169 ALR 227
Case Text: view of such authority, there can be no scope for the Court to reconsider the questions it has already answered unless it is found

Case ID: Case1004
Case Title: Mutual Pools &amp;amp; Staff Pty. Limited v The Commonwealth of Australia [1994] HCA 9 ; (1994) 179 CLR 155
Case Text: the reasons in Tran No. 1 at [50]-[54] the Court reiterates that no compensation is payable since s 51(xxxi) of the Constitution do

Case ID: Case1005
Case Title: NAGV and NAGW of 2002 v Minister for Immigration and Multicultural and Indigenous Affairs and Another [2005] HCA 6 ; (2005) 222 CL
```

- Search by case title

```
[51] query = "Barton v Croner Trading Pty Ltd (1984) 3 FCR 95"
     search_title(query, documents)
```

```
Search results:
Case ID: Case69
Case Title: Barton v Croner Trading Pty Ltd (1984) 3 FCR 95
Case Text: My own view is that s 21R confers a power to appoint, not a power to make, grant or issue any instrument. Section 21R does not talk i
```

- Wildcard query search
    1. Suffix query

```
[53] #suffix wildcard
     query = "*wealth"
     wildcard_query_search(inverted_index, documents, query)
```

```
Case ID: Case476
Case Title: R v King [2007] 2 NZLR 137
Case Text: It is sufficient for present purposes to begin by stating that in order to maintain a claim of legal professional privilege, a prop

Case ID: Case1004
Case Title: Mutual Pools &amp;amp; Staff Pty. Limited v The Commonwealth of Australia [1994] HCA 9 ; (1994) 179 CLR 155
Case Text: the reasons in Tran No. 1 at [50]-[54] the Court reiterates that no compensation is payable since s 51(xxxi) of the Constitution do

Case ID: Case480
Case Title: Southern Equities Corporation Ltd v West Australian Government Holdings Ltd (1993) 10 WAR 1
Case Text: It is sufficient for present purposes to begin by stating that in order to maintain a claim of legal professional privilege, a prop

Case ID: Case1006
Case Title: Newcrest Mining (WA) Limited and Another v The Commonwealth of Australia and Another [1997] HCA 38 ; (1997) 190 CLR 513
Case Text: the reasons in Tran No. 1 at [50]-[54] the Court reiterates that no compensation is payable since s 51(xxxi) of the Constitution do

Case ID: Case1008
Case Title: Olbers Co Ltd v Commonwealth and Another [2004] FCAFC 262 ; (2004) 143 FCR 449
Case Text: appeal (see Olbers Co Ltd v Commonwealth and Another [2004] FCAFC 262 ; (2004) 143 FCR 449) the Full Court explicitly adopted the f

Case ID: Case1009
Case Title: Olbers Co Ltd v Commonwealth of Australia and Another [2004] FCA 229 ; (2004) 136 FCR 67
Case Text: Olbers Co Ltd v Commonwealth of Australia and Another [2004] FCA 229 ; (2004) 136 FCR 67 at 79, French J, referring to s 106A of th

Case ID: Case486
Case Title: Westgold Resources NL v St Barbara Mines Ltd [2007] WASC 47
Case Text: It is sufficient for present purposes to begin by stating that in order to maintain a claim of legal professional privilege, a prop

Case ID: Case1012
```

2.Prefix query

```
[54] #prefix wildcard
     query = "heal*"
     wildcard_query_search(inverted_index, documents, query)
```

```
Search results:
Case ID: Case555
Case Title: Thomas v School of Mines Services Pty Ltd (unreported, Industrial Relations Court of Australia, Farrell JR, 5 August 1996)
Case Text: In support of a narrow construction of s 824(2), the respondents referred me to several authorities that had considered the former

Case ID: Case556
Case Title: Turner v Composite Buyers Ltd (unreported, Industrial Relations Court of Australia, Millane JR, 28 January 1997)
Case Text: In support of a narrow construction of s 824(2), the respondents referred me to several authorities that had considered the former

Case ID: Case557
Case Title: Fenech v Perfect Health Medical Centres Pty Ltd (unreported, Industrial Relations Court of Australia, Ryan JR, 12 June 1998)
Case Text: In support of a narrow construction of s 824(2), the respondents referred me to several authorities that had considered the former

Case ID: Case692
Case Title: Aldi Stores Limited Partnership v Frito-Lay Trading GmbH (2001) 54 IPR 344
Case Text: The same differences are created by an aural comparison. Here, stress and pronunciation would usually given to the first syllable o

Case ID: Case693
Case Title: Aristoc Limited v Rysta Limited (1944) 62 RPC 65
Case Text: The function of a trade mark is to give an indication to a purchaser as to the manufacture or quality of the goods and an indicatio

Case ID: Case694
Case Title: Australian Co-operative Foods v Norco Co-operative [1999] NSWSC 274 ; (1999) 46 NSWLR 267
Case Text: In Australian Co-operative Foods v Norco Co-operative [1999] NSWSC 274 ; (1999) 46 NSWLR 267 at [84] [86], Bryson J took a wide vie

Case ID: Case695
Case Title: Australian Woollen Mills Limited v FS Walton &amp;amp; Co Limited [1937] HCA 51 ; (1937) 58 CLR 641
Case Text: These principles are well established. They were stated by Dixon and McTiernan JJ more than 50 years ago in Australian Woollen Mill

Case ID: Case696
```

- Phrase query

```
[56] query = 'free rider'
     phrase_query_search(documents, query)
```

```
Search results:
Case ID: Case455
Case Title: Commissioner of Australian Federal Police v Propend Finance Pty Ltd [1997] HCA 3 ; (1997) 188 CLR 501
Case Text: With respect to advice privilege it was, until recently, uncontroversial that the proponent had to establish three elements: (1) a co

Search results:
Case ID: Case463
Case Title: Hickman v Taylor (1946) 329 US 495
Case Text: Second, the "free rider" argument fails because it is, ultimately, founded on a theory of "work product" privilege not accepted in Au

Search results:
Case ID: Case475
Case Title: Public Transport Authority of Western Australia v Leighton Contractors Pty Ltd [2007] WASCA 151
Case Text: It is sufficient for present purposes to begin by stating that in order to maintain a claim of legal professional privilege, a propone
```

- Boolean Search

```
boolean_query_search('Health And Ageing', inverted_index, doc_ids, documents,)
```

```
Case ID: Case79
Case Title: Nguyen v Minister for Health and Ageing [2002] FCA 1241
Case Text: The applicants relied upon the decisions in Nguyen v Minister for Health and Ageing [2002] FCA 1241 and Registrar for Liquor Licences

Case ID: Case83
Case Title: Registrar for Liquor Licences v Iliadis (1988) 19 FCR 311
Case Text: The applicants relied upon the decisions in Nguyen v Minister for Health and Ageing [2002] FCA 1241 and Registrar for Liquor Licences
```

- Stemming of data

```
new_mov["tags"]=new_mov["tags"].apply(stem)
new_mov
```

```
<ipython-input-23-7c817c95d79a>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  new_mov["tags"]=new_mov["tags"].apply(stem)
```

	case_id	case_title	tags
0	Case1	[Alpine, Hardwood, (Aust), Pty, Ltd, v, Hardys...	alpin hardwood (aust) pti ltd v hardi pti ltd ...
1	Case2	[Black, v, Lipovac, [1998], FCA, 699, ;, (1998...	black v lipovac [1998] fca 699 ; (1998) 217 al...
2	Case3	[Colgate, Palmolive, Co, v, Cussons, Pty, Ltd,...	colgat palmol co v cusson pti ltd (1993) 47 fc...
3	Case4	[Dais, Studio, Pty, Ltd, v, Bullett, Creative,...	dai studio pti ltd v bullett creativ pti ltd [...
4	Case5	[Dr, Martens, Australia, Pty, Ltd, v, Figgins,...	dr marten australia pti ltd v figgin hold pti ...
...	...	...	...
995	Case1014	[R, v, Oblach, [2005], NSWCCA, 440, ;, (2005),...	r v oblach [2005] nswcca 440 ; (2005) 65 nswlr...
996	Case1015	[Re, Centennial, Coal, Co, Ltd, (2006), 226, A...	re centenni coal co ltd (2006) 226 alr 341 mea...
997	Case1016	[Re, Director, of, Public, Prosecutions;, Ex, ...	re director of public prosecutions; ex part la...
998	Case1017	[Re, Venice, Nominees, Pty, Ltd, (Receiver, an...	re venic nomine pti ltd (receiv and manag appo...
999	Case1018	[Rutu, and, Another, v, Dalla, Costa, (1997), ...	rutu and anoth v dalla costa (1997) 139 flr 26...

997 rows × 3 columns

- Transforming to array

```
[28] temp=temp.fit_transform(new_mov["tags"]).toarray()
     temp
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

- Finding the cosine similarity

```
[36] sim=cosine_similarity(temp)
```

```
[37] sim
```

```
array([[1.        , 0.14679696, 0.88366859, ..., 0.0323196 , 0.02659618,
        0.01105398],
       [0.14679696, 1.        , 0.13380565, ..., 0.04440912, 0.02645278,
        0.03471443],
       [0.88366859, 0.13380565, 1.        , ..., 0.03057455, 0.0251051 ,
        0.01134644],
       ...,
       [0.0323196 , 0.04440912, 0.03057455, ..., 1.        , 0.05743856,
        0.10998161],
       [0.02659618, 0.02645278, 0.0251051 , ..., 0.05743856, 1.        ,
        0.22444625],
       [0.01105398, 0.03471443, 0.01134644, ..., 0.10998161, 0.22444625,
        1.        ]])
```

- Retrieving top 5 cases similar to Case -2

```
[39] #to retrieve top 5 cases relevant to case 2
     retrieve("Case2",5)
```

```
['Case4', 'Case6', 'Case179', 'Case648', 'Case574']
```

- Retrieving top 10 cases similar to Case -7

```
[41] retrieve("Case7",10)
```

```
['Case5', 'Case146', 'Case536', 'Case972', 'Case969', 'Case701', 'Case580', 'Case987', 'Case981', 'Case86']
```